

HOJA 1: ANÁLISIS DE IMPACTO Y MODELO ARISTOTÉLICO DEL SOFTWARE

Premisa inicial: Se discute *cuánto de software se genera para la organización y cuánto se automatiza*. **No se discute si implementar software;** eso ya es un hecho consolidado.

"No Hay Balas de Plata" (Fred Brooks) — Modelo Aristotélico del Software

Los errores que el creador de software de IBM cometió al desarrollar el proyecto del 1^{er} Sistema Operativo se siguen cometiendo décadas después. Brooks distingue entre dos tipos de complejidades:

1. Dificultades Accidentales vs. Esenciales

- **Dificultades Accidentales:** Circunstancias del entorno o del proceso (ej. la gente se enferma, cambios imprevistos en requerimientos, limitaciones en horas de trabajo, problemas de Wi-Fi/conectividad, herramientas).
Nota: Han existido muchos intentos por solucionar las problemáticas del desarrollo de software atacando principalmente las características accidentales, pero solucionar estas no da grandes avances. Se debe focalizar en la parte esencial.
- **Dificultades Esenciales (Las 4 características intrínsecas de la esencia del SW):**
 1. **Complejidad e Invisibilidad:** El software es intrínsecamente complejo, y a esta característica contribuye fuertemente la *de invisibilidad* (se trabaja con ideas, cosas abstractas que no se pueden ver físicamente). El **60% del software se tira**; por más que haya grandes avances tecnológicos, el problema raíz sigue siendo la comunicación entre personas (**¿Qué se va a crear?**).
 2. **Mutabilidad:** El software da la falsa idea de que es fácil de modificar ("solo tocar código"), pero al momento de cambiar requerimientos, incorporar esos cambios no es tan sencillo: muchos de ellos impactan directamente en la arquitectura. Asociado a esto, a medida que el software se va cambiando se aumenta el riesgo de romper cosas, comienzan los malos funcionamientos y a largo plazo se complica el mantenimiento y seguimiento.
 3. **Conformidad:** Crear software es una actividad de servicio orientada a satisfacer las necesidades de alguien, lo que vuelve sumamente importante la conformidad del cliente con el producto final.
 4. **Invisibilidad:** La falta de presencia geométrica/espacial dificulta su visualización y control integral.

2. La Inteligencia Artificial y las "Balas de Plata"

Aparecieron innovaciones que significaron avances importantes pero **no son balas de plata** (como Scrum, POO, etc.). Actualmente, la bala de plata parece ser la **IA**:

- **El cuello de botella actual:** Es para el ser humano **definir qué crear**. Ahora el humano piensa y decide, genera el prompt, y la IA se encarga de la creación con un alto nivel de eficiencia.
- La IA ayuda mucho con la **parte accidental** (generación rápida de código, sintaxis), pero **no soluciona lo esencial**: nos ayuda en el *cómo* (las distintas representaciones dentro del proceso iterativo de desarrollo), pero no en el *qué*.

Scrum y la Documentación (Apunte lateral)

Scrum le da poder al equipo sobre qué necesita documentar. Agiliza y marca la preferencia de tener software funcionando, **pero siempre hay que documentar**.

3. ¿Qué hace que los proyectos de desarrollo de SW sean exitosos?

- Que el usuario se involucre activamente.
- Que la gerencia apoye firmemente el proyecto.
- Enunciado claro de los requerimientos.
- Planeamiento adecuado y expectativas realistas.
- Hitos intermedios bien definidos.
- Personas involucradas competentes.

Sólo saber programar, no alcanza.

HOJA 2: LEAN, AGILE VS. TRADICIONAL

1. Conceptos Fundamentales de Ingeniería de Software

- **Ingeniería:** Conjunto de conocimientos, herramientas y métodos que encaran un problema para darle solución.
- **Ingeniería de Software:** Disciplina de la ingeniería que se ocupa de *todos* los aspectos de la producción de software, desde su concepción inicial, hasta su mantenimiento y despliegue en funcionamiento.
- **Cuerpo de Conocimientos: SWEBOK** (*Software Engineering Body of Knowledge*), Versión 3.0 del 2014 de la IEEE.

2. Contexto de la Ingeniería de Software (3 Disciplinas)

Disciplina	Áreas y Actividades Incluidas
Técnicas	Requerimientos, análisis y diseño, construcción, prueba, despliegue (<i>marcado en apunte: # no práctico [?]</i>).
De Gestión	Planificación de proyecto, monitoreo y control de proyectos.
De Soporte	Gestión de Configuración de Software (SCM), aseguramiento de calidad (QA), métricas.

3. SW Configuration Management (SCM) - Introducción

La disciplina de SCM **no es lo mismo que usar una herramienta de gestión de versionado** (Git, Drive, SVN, DevOps). *Usar una herramienta no significa que se siga la disciplina.*

4. Tipos de Procesos: Definidos vs. Empíricos

Existen 2 grandes tipos de procesos (tanto *Lean* como *Agile* se basan en los empíricos; Scrum es un referente de Agile que es un framework que usa procesos empíricos):

Personas

Empirismo: Es experimentar y aprender en base a las experiencias. Se le da poder al equipo sobre qué proceso usar; aprendés de experiencias para *redirigir o consolidar*.

5. Anotaciones Marginales y Detalle en Lápiz: Procesos vs. Personas

LÁPIZ: PROCESO **Procesos Definidos (Tradicional / PVD):** La empresa busca tener previsibilidad; asume que se puede ser predecible si somos repetibles. Un proyecto antiguo marca las pautas de tiempo y otras cosas respecto a proyectos nuevos a emprender. *Cometen el error de traspolar experiencias de un equipo/proyecto a otro cuando son equipos y proyectos totalmente diferentes.* En los procesos definidos desarrollan y documentan la máxima cantidad de cosas para un proyecto y luego este proceso se adapta para un proyecto en particular ("**se sacan cosas**").

LÁPIZ: PROCESO **Enfoque Agile:** Agile y otros afirman que **la experiencia no es traspolable**, por lo que cada equipo y proyecto son particulares.

LÁPIZ: PERSONAS **Variabilidad del Factor Humano:** Los equipos no son iguales: existen diferencias profundas de conocimiento, cultura organizacional y ubicación geográfica.

LÁPIZ: HERRAMIENTAS **Automatización:** Con las herramientas se busca automatizar lo más que se pueda en los proyectos. En cada empresa se evalúa cuáles están disponibles según las políticas de cada empresa.

HOJA 3: GESTIÓN DE CONFIGURACIÓN DE SOFTWARE (SCM)

1. Conceptos Nucleares de Configuración

- **Configuración del Software:** Es como una "foto" en un momento en el tiempo de todos los ítems de configuración que están guardados en un repositorio.
- **Ítem de Configuración (SCI):**
 - Para que algo sea un ítem de configuración, debe poder guardarse en una computadora (digital). Si algo está en papel, para que sea un ítem de configuración **se debe digitalizar**.
 - Puede ser: una minuta de reunión, un caso de uso (CU), requerimientos, código fuente, etc.
 - Es el estado de software que se **identifica unívocamente** mediante número y versión.
- **Herramientas de Versionado:** Son las que ayudan grandemente para moverse entre las versiones del software, realizar cambios trazables y poder revertir estos cambios si fallan.

2. ¿Por qué cambia el Software? (Cambios Constantes)

El software está sometido a cambios constantes debido a:

1. **Cambios en el negocio** o nuevos requerimientos del cliente.
2. **Cambios internos (Deuda Técnica):** Mejoras internas, refactorización o casos que quedaron incompletos.
3. **Cambios en herramientas / tecnología:** Modificación obligada de tecnologías (ej. cambiar una Base de Datos completa o actualizar dependencias/frameworks).

3. Definición según IEEE 828 y Objetivos de SCM

Estándar IEEE 828 - SCM como Actividad Paraguas

SCM es una **actividad paraguas transversal** a lo largo de todo el ciclo de vida del proyecto. Se necesita una buena administración de la configuración de software para brindar un lugar (**contenedor**) de los ítems con la configuración ordenada, de forma que se realice de manera constante un control riguroso, asegurando la calidad del producto final entregado.

Se necesita esta disciplina de gestión de configuración debido a la **volatilidad del producto**, de manera que se mantenga la integridad de este:

Pilar	Descripción y Alcance
Integridad	Garantiza que el producto satisface fielmente las necesidades del cliente y mantiene su consistencia sin pérdidas.
Trazabilidad	Se debe poder responder preguntas sobre el porqué de cada elemento que existe en el proyecto/ producto (origen y justificación del cambio).
Control de Componentes	Resuelve los problemas comunes en el manejo de componentes: • Pérdida de componentes en sí o de los cambios realizados sobre ellos. • Inclusión de cambios no válidos o no autorizados. • Superposición destructiva de cambios entre desarrolladores concurrentes.

Fin de los apuntes de la clase del 13/08/2026 — Transcripción íntegra y completa.